

CNN-Based Crop Disease Diagnosis with Retrieval-Driven LLM Support

1st Abhinav Yadav

Department of CSIT

KIET Group of Institutions

Ghaziabad, India

abhinavat3791@gmail.com

2nd Mr. Yousuf Haider

Department of CSIT

KIET Group of Institutions

Ghaziabad, India

yousuf.haider@kiet.edu

3rd Ayush Katiyar

Department of CSIT

KIET Group of Institutions

Ghaziabad, India

ayush.2226csit1040@kiet.edu

4th Anurag Kumar

Department of CSIT

KIET Group of Institutions

Ghaziabad, India

anurag.2226csit1001@kiet.edu

Abstract—CropCure AI is a hybrid intelligent system built to diagnose plant diseases and support agricultural decision-making end-to-end. It combines a lightweight custom CNN for image-based disease classification with a Retrieval-Augmented Generation (RAG) pipeline using MiniLM embeddings and a FAISS similarity index. Groq Llama-3.3-70B generates context-aware treatment recommendations for complex queries, while routine inputs are handled by a lightweight local LLM to reduce latency and API overhead. OpenRouter GPT supports a standalone indoor plant advisory module for home gardening guidance.

The CNN achieves 93.06% test accuracy across 15 disease classes, demonstrating reliable generalization under real-world conditions. By pairing deep learning-based visual diagnosis with retrieval-grounded language reasoning, CropCure AI delivers interpretable, actionable guidance that remains computationally efficient for resource-constrained environments.

Index Terms—Plant disease detection, Convolutional Neural Network, Retrieval-Augmented Generation, FAISS, LLM, Groq Llama, OpenRouter, Agriculture AI, Deep Learning, Precision Agriculture.

I. INTRODUCTION

Agriculture feeds the world and drives economies, making timely crop disease detection a pressing need. Small farmers, especially in developing regions, often rely on expensive or hard-to-reach experts. Although photo-based disease identification has come a long way, most tools simply flag the problem without telling farmers what to actually do about it.

Over the years, farming technology shifted from simple rule-based programs to more sophisticated deep learning approaches. Even so, very few tools manage to both read images well and offer smart, step-by-step guidance. Most existing solutions handle only one side of the problem, spotting issues in the field but leaving farmers without a clear path forward.

CropCure AI was built to close this gap. It brings together detection, knowledge retrieval, and language reasoning into one connected workflow, giving farmers not just a diagnosis but the practical advice they need to act on it.

- A purpose-built CNN that identifies plant leaf diseases from images with high precision, designed without unnecessary complexity.
- A FAISS-powered RAG setup that grounds every response in retrieved agricultural data, avoiding unreliable or fabricated advice.

- A multi-LLM layer where Groq Llama-3.3-70B handles complex queries, Mini-LLM manages routine conversations, and OpenRouter GPT supports specialised consultations.
- A lightweight Flask backend connecting disease detection to a conversational UI, delivering personalised farming guidance in real time.

Beyond simple diagnosis, the system provides farmers with clear, tailored advice covering routine care, prevention measures, and access to local support when needed.

The key contributions of this work are:

- 1) A hybrid CNN-RAG-LLM architecture purpose-built for agricultural use, rather than adapted from general-purpose AI systems.
- 2) A structured multi-LLM pipeline that maintains both response accuracy and low inference latency.

II. RELATED WORK

Deep learning has gained significant traction in agricultural disease detection due to its ability to recognize visual symptoms directly from images.

Hu et al. proposed a compressed ResNet for IoT deployment enabling real-time diagnosis, while Patil and Kumar developed Rice-Fusion combining sensor and image data for rice disease detection together highlighting how architecture optimization and data fusion advance crop disease identification.

Li et al. and Balafas et al. demonstrated that CNN architectures such as ResNet, DenseNet, and Inception outperform classical methods in disease classification. However, their practical deployment remains limited most lack real-time capability and integration with farmer-facing decision tools.

Joseph et al. improved detection precision using real-time field-collected data, validating model performance beyond controlled settings. Alharbi et al. emphasized incremental model adaptation to handle emerging crop diseases without losing prior knowledge.

Chozo et al., Shruhi et al., and Rameesh et al. further validate CNN effectiveness in plant disease detection, while Mishra et al. extended this to real-time maize crop monitoring, demonstrating efficiency in field conditions.

Despite these advances, most existing systems focus solely on detection or classification, offering little actionable guid-

ance. Research tends to prioritize accuracy metrics while overlooking practical farmer needs such as treatment steps, prevention advice, and contextual crop information.

Few systems combine image-based disease detection with automated, knowledge-grounded recommendations. Visual analysis and text-based advisory typically exist as separate components, rarely unified in a single pipeline.

Our work addresses this gap by pairing a lightweight CNN for disease detection with a RAG-enhanced language model that retrieves relevant agricultural knowledge, bridging the divide between what the model identifies and the guidance farmers actually need.

III. DATASET AND PREPROCESSING

A. Dataset Composition

Sourced from PlantVillage, open farm records, and real plant images, the dataset contains over 20,000 images across multiple crop types, with 3,096 held-out images (15%) used for evaluation.

The dataset covers tomatoes, potatoes, and peppers, each including a healthy class for contrast:

- **Tomato:** Early Blight, Late Blight, Leaf Mold, Bacterial Spot, Healthy
- **Potato:** Early Blight, Late Blight, Healthy
- **Pepper:** Bacterial Spot, Healthy
- **Additional Classes:** Further disease categories for the same crops.

Training across varied lighting, leaf orientations, cluttered backgrounds, and disease stages ensures robust performance in real farm conditions.

B. Data Preprocessing Pipeline

A full preprocessing pipeline was established to maintain input quality across both system stages: leaf detection and disease classification.

1) *Image Standardization:* All images were resized to a uniform resolution to reduce noise from size variation and improve batch processing efficiency.

- **Leaf Verification CNN:** Images resized to 128×128 pixels and converted to normalized tensors for binary classification.
- **Disease Classification CNN:** Images resized to 256×256 pixels with normalized statistics to improve training stability.

Pixel values were rescaled to $[0, 1]$ followed by model-specific normalization to stabilize gradients and prevent numerical issues during training.

2) *Color Space Processing:* All images were converted to RGB before CNN input, ensuring color space consistency across samples.

3) *Data Augmentation:* Random augmentation during training included horizontal and vertical flips, rotations, color jitter, affine transforms, random crops, and light Gaussian noise collectively improving generalization to real field conditions.

These techniques increase input diversity without requiring additional images, improving model robustness across real-world conditions.

$$\text{Augmented Image} = \mathcal{T}_{\text{rotation}} \circ \mathcal{T}_{\text{flip}} \circ \mathcal{T}_{\text{color}} \circ \mathcal{T}_{\text{crop}}(I_{\text{original}}) \quad (1)$$

C. Data Splitting Strategy

The dataset was divided using stratified sampling to preserve class distribution across all subsets:

- **Training Set (70%):** Used for model training and parameter optimization.
- **Validation Set (15%):** Used for hyperparameter tuning and early stopping.
- **Test Set (15%):** Used for unbiased evaluation of generalization performance.

This stratified split ensures balanced class representation in each subset, yielding reliable and fair performance estimates.

IV. PROPOSED CNN ARCHITECTURE

A. Architecture Design Principles

The custom CNN targets efficiency on resource-constrained devices while preserving detection across fine grained disease markers and varied imaging conditions.

B. Model Architecture Details

Four convolutional layers followed by a compact fully connected classifier, totaling 424,207 trainable parameters.

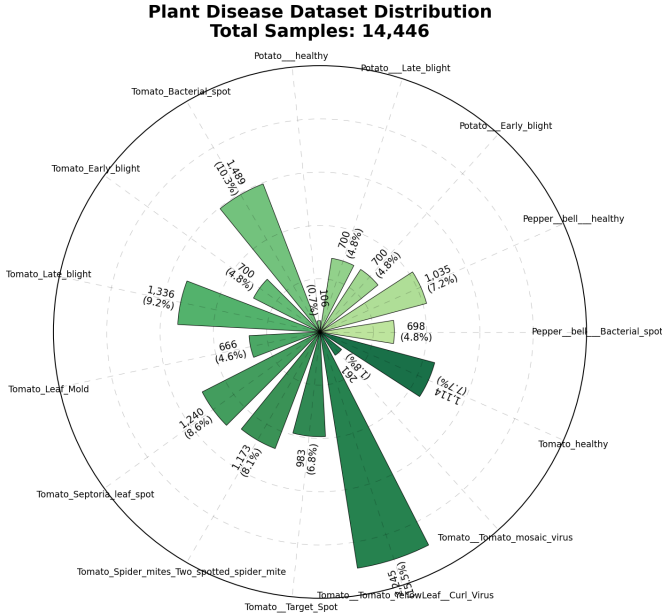


Fig. 1: **Dataset class distribution:** balanced representation across the main crop types across the 15 disease categories.

```

class PlantDiseaseModel(nn.Module):
    def __init__(self, num_classes):
        super(PlantDiseaseModel, self).__init__()

        self.conv_block1 = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding="same"),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.conv_block2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, padding="same"),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.conv_block3 = nn.Sequential(
            nn.Conv2d(64, 128, kernel_size=3, padding="same"),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.conv_block4 = nn.Sequential(
            nn.Conv2d(128, 256, kernel_size=3, padding="same"),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )

        self.global_avg_pool = nn.AdaptiveAvgPool2d((1, 1))

        self.fc_block = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.conv_block1(x)
        x = self.conv_block2(x)
        x = self.conv_block3(x)
        x = self.conv_block4(x)
        x = self.global_avg_pool(x)
        x = self.fc_block(x)
        return x

```

Fig. 2: CNN Architecture Implementation.

C. Training Methodology

Training focused on stable convergence and balanced performance across all classes, using dynamically adjusted techniques rather than fixed schedules.

Categorical cross-entropy was used as the loss function, appropriate for multi-class single-label classification:

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(p_i) \quad (2)$$

where y_i is the true one-hot label and p_i is the predicted probability for class i .

SGD was selected for parameter updates with a learning rate of $\eta = 2 \times 10^{-3}$:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}$$

Training Setup:

- Class count determined automatically via LabelEncoder.
- Device: CUDA GPU if available, otherwise CPU.
- Consistent batch size across training, validation, and test sets; maximum 100 epochs.

Early stopping was applied with a patience of 4 epochs and minimum improvement threshold of 0.01. The best-performing validation checkpoint was saved as `best_model.pth`, with

gradients updated via SGD and progress tracked after each epoch.

D. Evaluation of Performance

Training ran for up to 100 epochs with early stopping applied once validation performance plateaued. Final evaluation on the held-out test set demonstrated consistent results across all key metrics.

TABLE I: Performance Metrics of Proposed CNN Architecture

Metric	Value
Test Accuracy	93.06%
Macro Precision	94%
Macro Recall	91%
Macro F1-Score	92%
Weighted Precision	93%
Weighted Recall	93%
Weighted F1-Score	93%

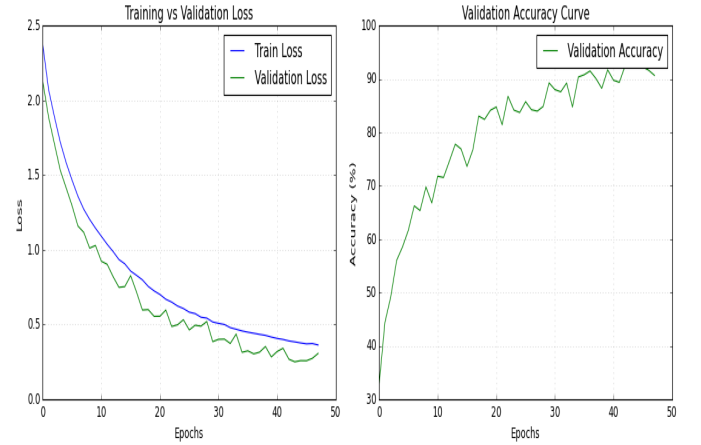


Fig. 3: Illustrates the improvements in training and validation over time

The predicted disease label is passed directly to the RAG pipeline, which retrieves domain-specific knowledge to generate treatment and prevention recommendations connecting visual diagnosis to contextual reasoning within a single unified flow.

V. RAG PIPELINE AND LLM INTEGRATION

A. Knowledge Base Construction

The knowledge base (`crop_data.json`) covers six categories: disease profiles, symptom documentation, treatment protocols, prevention strategies, government subsidy schemes, and expert extension contacts. Each entry was reviewed for accuracy before inclusion, with unverified or regionally restricted recommendations excluded. Since RAG outputs are bounded strictly by retrieved content, knowledge base quality directly governs response reliability.

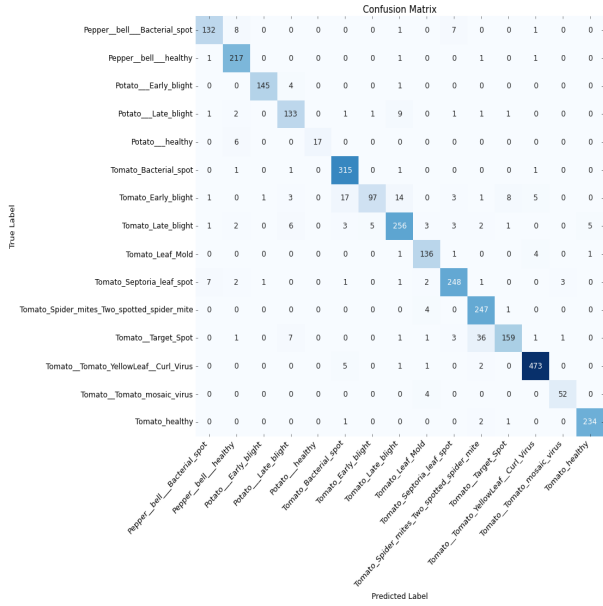


Fig. 4: Confusion matrix of all class disease classification strong diagonal dominance and few misclassification.

B. Embedding and Vector Indexing

Text entries are encoded into 384-dimensional vectors using SentenceTransformer MiniLM-L6-v2. A FAISS index is built as:

$$\text{Index} = \text{IndexFlatL2}(384) \quad (3)$$

L2 distance metric clusters semantically related agricultural concepts, enabling fast and accurate retrieval.

C. Mechanism of Retrieval

The system computes embedding q_e for query q and performs a similarity search:

$$\text{Retrieved Documents} = \text{FAISS.search}(q_e, k = 2) \quad (4)$$

The two closest matching documents are retrieved to provide the LLM with focused, relevant context.

D. LLM Inference Pipeline

FAISS retrieves the top- k documents, which are injected into the model prompt to generate grounded responses:

- 1) **Groq Llama-3.3-70B Versatile:** Primary reasoning component on Groq's inference platform, handling complex queries on plant diseases, environmental factors, and field interventions.
- 2) **Context-Guided Response Generation:** Retrieved documents are embedded into the prompt, anchoring outputs to verified agricultural data and reducing hallucinations.
- 3) **Lightweight Local LLM:** Routine queries are routed to a lightweight local model rather than Groq Llama, reducing API calls and latency. A keyword and confidence-threshold classifier directs disease-specific queries to Groq Llama-3.3-70B while handling general queries locally.

E. Efficient Engineering Approach

Each user query is concatenated with the top- k retrieved documents before being passed to the model:

$$\text{Prompt} = \text{Context}_{\text{retrieved}} \parallel \text{UserQuery} \quad (5)$$

This grounds responses strictly in retrieved agricultural data, preventing hallucinations by restricting the model to domain-relevant context.

Listing 1: RAG Prompt Template

You are an agriculture assistant. Answer based only on this context:
 {retrieved_context}
 Question:
 {user_query}

The combined prompt is passed directly to Groq Llama-3.3-70B, ensuring factually grounded and domain-constrained responses.

VI. SYSTEM ARCHITECTURE

CropCure AI adopts a modular architecture where each component manages a distinct phase of plant analysis and response generation, enabling scalability, easier maintenance, and efficient handling of both image and text inputs.

A. Data Flow

- 1) **User Input:** A plant leaf image or text query is submitted.
- 2) **Leaf Verification:** A lightweight CNN validates the image as a leaf; invalid inputs are rejected with a resubmission prompt.
- 3) **Disease Classification:** Verified images are classified, returning the disease class, confidence score, and top-5 probabilities.
- 4) **RAG Retrieval:** The query is embedded via MiniLM and matched against the FAISS index to retrieve relevant symptoms, treatments, and prevention data.
- 5) **LLM Response Generation:** Groq Llama-3.3-70B generates treatment recommendations from retrieved context; OpenRouter GPT handles indoor plant queries.
- 6) **Response Delivery:** Disease prediction, treatment advice, and prevention recommendations are returned to the user interface.

VII. BACKEND IMPLEMENTATION

The backend uses a Flask REST architecture, integrating CNN models, FAISS retrieval, and external LLM services for model loading, inference, request routing, and JSON response generation.

A. Flask REST API Endpoints

Four endpoints handle core functions — disease classification, chat, indoor plant recommendations, and leaf verification:

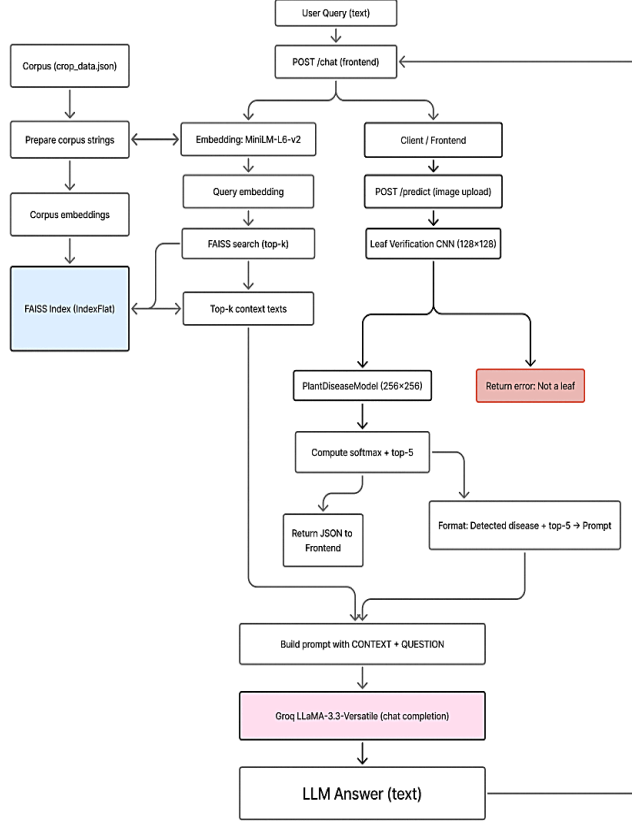


Fig. 5: Overall system architecture showing the flow between leaf verification, disease classification, RAG retrieval, LLM reasoning, and the Flask backend API.

Listing 2: Flask REST API Endpoints

```

@app.route('/predict', methods=['POST'])
def predict(): pass
@app.route('/chat', methods=['POST'])
def chat(): pass
@app.route('/indoor-plants/recommend', methods=['POST'])
def indoor_plants_recommend(): pass
@app.route('/test-leaf', methods=['POST'])
def test_leaf_verification(): pass

```

B. Leaf Verification System

A PyTorch CNN filters non-leaf inputs prior to classification. Images are resized, normalized, and evaluated via sigmoid activation:

$$\hat{y} = \sigma(f_{\text{CNN}}(I)) \quad (6)$$

where I is the input image, f_{CNN} is the verification model, and σ is the sigmoid function. Invalid inputs are rejected with a user notification.

C. Disease Classification Pipeline

Verified images are resized to 256×256 with ImageNet normalization and passed to a PyTorch classifier, which outputs

the predicted class, confidence score, and top-5 probabilities mapped to human-readable labels.

D. RAG Knowledge Retrieval System

Documents are embedded via MiniLM-L6-v2 (384-dimensional vectors) and indexed using FAISS IndexFlatL2. At query time, top- k relevant entries are retrieved and injected into the LLM prompt to ensure grounded, factual responses.

E. LLM Reasoning Layer

Two external LLM services are integrated:

- **Groq Llama-3.3-70B**: Handles disease reasoning, treatment suggestions, and contextual explanations via `/chat`.
- **OpenRouter-GPT**: Generates structured indoor plant care recommendations via `/indoor-plants/recommend`.

F. Backend Responsibilities

The Flask server manages CNN and embedding model loading, FAISS-based retrieval, external LLM API calls, JSON response formatting, and fallback error handling.

VIII. INDOOR PLANT RECOMMENDATION MODULE

This module serves urban indoor gardeners, using OpenRouter's AI models to deliver personalized recommendations covering cultivation techniques, care schedules, placement guidance, health benefits, and troubleshooting:

Listing 3: Indoor Plant Prompt Structure

```

- User experience level: {experience}
- Available light: {light_conditions}
- Space constraints: {space_limits}
- Maintenance preference: {maintenance_level}
Return structured JSON with:
1. plant_recommendations (array)
2. care_instructions (object)
3. common_issues (array)
4. benefits (array)

```

Structured JSON responses are delivered through the UI, providing users with fast, actionable indoor gardening advice.

IX. EXPERIMENTAL RESULTS AND DISCUSSION

A. CNN Classification Performance

The proposed CNN achieved **93.06%** test accuracy, outperforming all baseline models while maintaining low computational overhead suitable for practical deployment.

TABLE II: Comparative Analysis with Baseline Models

Model	Accuracy	Parameters	Inference Time	F1-Score
Proposed CNN	93.06%	424K	45ms	92%
ResNet-50	89.23%	23.5M	120ms	88.75%
MobileNetV2	87.56%	2.2M	65ms	86.91%
VGG-16	90.12%	134.3M	180ms	89.45%
EfficientNet-B0	91.34%	4.0M	85ms	90.87%

Integrating CNN-based vision with a RAG-powered LLM produces a unified system that improves accuracy, delivers

data-backed explanations, and provides transparent, context-aware recommendations tailored to real-world farming needs.

B. Sample System Outputs

Cases illustrate pipeline behavior across disease-positive and negative scenarios.

Case 1: Healthy Pepper (97.2%): No treatment fabricated on disease-free input; preventive care summary returned instead.

Case 2: Potato Early Blight (91.7%): Concentric ring patterns correctly classified. System surfaced a chlorothalonil spray schedule, rotation advice, and a linked state fungicide subsidy.

C. Limitations and Challenges

Several limitations remain. Knowledge base accuracy depends on regular expert review, and CNN misclassification can propagate errors downstream. LLM inference requires stable connectivity, degrading performance in offline scenarios. Recommendations may need local adaptation given geographic variability, and adequate processing resources remain a practical requirement despite architectural optimization.

X. CONCLUSION AND FUTURE WORK

CropCure AI demonstrates that lightweight CNN-based visual diagnosis and retrieval-augmented language reasoning can be unified into a practical, farmer accessible decision support system. Leaf verification, disease classification, knowledge retrieval, and multi-LLM orchestration operate as a single connected pipeline, keeping the workflow transparent and accessible to non-technical users. Key contributions include a CNN achieving 93.06% test accuracy across 15 disease classes, FAISS-backed retrieval grounding every LLM response in verified agricultural data, and a multi-LLM routing layer that balances response quality against inference latency all packaged into a deployable, farmer oriented system.

A. Ethical Considerations

CropCure AI does not store or transmit user-submitted images beyond the active session, preserving farmer data privacy. System outputs are intended to supplement, not replace, agronomist advice, particularly for severe or unfamiliar disease cases. Developers and deployers should remain aware that misclassification risk exists, and that recommendations may require local adaptation before acting on them.

B. Future Work

- **Advanced Vision:** Exploring vision transformers for finer-grained disease feature detection.
- **Multi-Label Classification:** Extending the CNN to handle co-occurring infections on a single leaf.
- **Edge Deployment:** Enabling offline functionality for low-connectivity farming environments.
- **Knowledge Base Expansion:** Adding expert-verified data and real-time disease alert integration.
- **Adversarial Robustness:** Implementing input sanitization and retrieval validation against misleading content.

- **Feedback-Based Learning:** Adapting recommendations progressively through farmer feedback loops.

ACKNOWLEDGMENT

We thank KIET Group of Institutions for providing the resources and environment that made this project possible, and our mentors and subject matter experts for their guidance throughout the design, testing, and evaluation phases.

REFERENCES

- [1] Z. Hossain, M. M. Khan, S. Thapa, R. Uddin, E. J. Meem, S. K. Niloy, M. A. Rafi, M. S. J. Shazib, M. F. Kabir, and G. D. Bhavani, "Advanced Deep Learning Techniques for Precision Diagnosis of Tea Leaf Diseases," in *Proc. IEEE Int. Conf. Emerging Technologies and Applications (MPSec ICETA)*, pp. 1–6, 2025.
- [2] A. Iqbal, "PlantHealthNet: Transformer-Enhanced Hybrid Models for Disease Diagnosis and Severity Estimation in Agriculture," *IEEE Access*, 2025.
- [3] M. E. Rayed, J. R. Jim, M. J. Islam, M. F. Mridha, M. M. Kabir, and M. J. Hossen, "MangoLeafXNet: An Explainable Deep Learning Model for Accurate Mango Leaf Disease Classification," *IEEE Access*, 2025.
- [4] M. S. A. M. Al-Gaashani, R. Alkanhel, M. A. S. Ali, M. S. A. Muthanna, A. Aziz, and A. Muthanna, "MSCPnet: A Multi-Scale Convolutional Pooling Network for Maize Disease Classification," *IEEE Access*, 2025.
- [5] S. N. Yousafzai, I. M. Nasir, S. Tehsin, D. S. Malik, I. Keshta, N. L. Fitriyani, Y. Gu, and M. Syafrudin, "Multi-Stage Neural Network-Based Ensemble Learning Approach for Wheat Leaf Disease Classification," *IEEE Access*, 2025.
- [6] A. Bhargava, A. Shukla, O. P. Goswami, M. H. Alsharif, P. Uthansakul, and M. Uthansakul, "Plant Leaf Disease Detection, Classification, and Diagnosis Using Computer Vision and Artificial Intelligence: A Review," *IEEE Access*, vol. 12, pp. 37443–37469, 2024.
- [7] D. S. Joseph, P. M. Pawar, and K. Chakradeo, "Real-time plant disease dataset development and detection of plant disease using deep learning," *IEEE Access*, vol. 12, pp. 16310–16333, 2024.
- [8] V. Balafas, E. Karantoumanis, M. Louta, and N. Ploskas, "Machine learning and deep learning for plant disease classification and detection," *IEEE Access*, vol. 11, pp. 114352–114377, 2023.
- [9] A. Alharbi, M. U. G. Khan, and B. Tayyaba, "Wheat disease classification using continual learning," *IEEE Access*, vol. 11, pp. 90016–90026, 2023.
- [10] L. Goel and J. Nagpal, "A systematic review of recent machine learning techniques for plant disease identification and classification," *IETE Technical Review*, vol. 40, no. 3, pp. 423–439, 2023.
- [11] R. R. Patil and S. Kumar, "Rice-fusion: A multimodality data fusion framework for rice disease diagnosis," *IEEE Access*, vol. 10, pp. 5207–5222, 2022.
- [12] L. Li, S. Zhang, and B. Wang, "Plant disease detection and classification by deep learning—A review," *IEEE Access*, vol. 9, pp. 56683–56698, 2021.
- [13] W.-J. Hu, J. Fan, Y.-X. Du, B.-S. Li, N. Xiong, and E. Bekkering, "MDFC-ResNet: An agricultural IoT system to accurately recognize crop diseases," *IEEE Access*, vol. 8, pp. 115287–115298, 2020.
- [14] S. Mishra, R. Sachan, and D. Rajpal, "Deep convolutional neural network based detection system for real-time corn plant disease recognition," *Procedia Computer Science*, vol. 167, pp. 2003–2010, 2020.
- [15] U. Shruthi, V. Nagaveni, and B. K. Raghavendra, "A review on machine learning classification techniques for plant disease detection," in *Proc. 5th Int. Conf. Advanced Computing Communication Systems (ICACCS)*, IEEE, 2019.
- [16] S. Ramesh et al., "Plant disease detection using machine learning," in *Proc. Int. Conf. Design Innovations for 3Cs Compute Communicate Control (ICDI3C)*, IEEE, 2018.
- [17] E. Rex, "Plant Disease Dataset," *Kaggle*, 2018. Available: <https://www.kaggle.com/datasets/emmarex/plantdisease>